FINAL PROTOTYPE STAGE REPORT

CityServe

Premium local services, made convenient for Thapar and Patiala

A local services marketplace web application

Next.js 14 · Express · Sequelize · MySQL

Team: Team Cyphers

Members: Ruhanika (1024030685)
Ayushi (1024030686)
Karan Bansal (1024030683)

Institution: Thapar Institute of Engineering and Technology (TIET)

Course: Software Engineering

Submitted to: Raghav B. Venkataramaiyer

Abstract

CityServe is a local services marketplace that helps residents and students in Patiala, Punjab discover, contact and book trusted local businesses such as electricians, plumbers, mechanics, tutors and laundries. The initial prototype demonstrated the core marketplace workflow with generic interface components and placeholder data. This report documents the complete system at the final prototype stage, with particular emphasis on the technical changes made since the initial prototype.

The final prototype integrates a dataset of 155 real Patiala businesses across 20 trade categories and 83 localities, sourced through the Google Places API, without modifying the existing MySQL schema. This was achieved with an in-memory dataset engine and hybrid backend controllers that merge database vendors with dataset records. The user interface was redesigned around an editorial design system, *The Local Standard*, and now offers locality-aware search, one-tap calling, directions, a map view and honest trust indicators such as "Not rated yet" in place of fabricated ratings. The report covers requirements, architecture, data model, implementation, verification results, challenges and their solutions, limitations and future work.

Keywords: local services marketplace, Next.js, Express, Sequelize, MySQL, data integration, trust and transparency, Patiala.

Contents

Abstract **i**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope of This Report 1

2 Requirements Analysis **1**

2.1 User Roles 1

2.2 Functional Requirements 2

2.3 Non-Functional Requirements 2

3 System Architecture **2**

3.1 Technology Stack 2

3.2 Three-Tier Design 2

3.3 Backend Module Structure 3

3.4 Data Model 3

3.5 Order Lifecycle 3

3.6 REST API Overview 4

3.7 Security Measures 4

4 Dataset **4**

4.1 Source and Schema 4

4.2 Coverage and Quality 5

5 Implementation of the Core System **5**

5.1 Authentication and Authorization 5

5.2 Catalog, Orders and Reviews 6

5.3 Frontend Pages 6

5.4 Demo Mode for External Services 6

6 Technical Changes Since the Initial Prototype **7**

6.1 Design Constraint: No Schema Changes 7

6.2 In-Memory Dataset Engine 7

6.3 Hybrid Backend Controllers 8

6.4 Editorial Design System: The Local Standard 8

6.5 Frontend Components and Pages 8

6.6 Theme Contrast Bug Fix 8

6.7 Data Honesty and Integrity Standards 9

7 Testing and Verification **9**

7.1 Production Build 9

7.2 API Verification 9

8 Deployment and Setup **10**

8.1 Prerequisites and Environment 10

8.2 Running the Application 10

9 Results and Discussion **10**

10 Conclusion **10**

10.1 Summary 11

10.2 Challenges Faced and How We Solved Them 11

10.3 Limitations 12

10.4 Future Work 12

References **12**

A Project Structure **12**

1 Introduction

1.1 Background and Motivation

Finding a dependable electrician, tutor or mechanic in a mid-sized city still relies largely on word of mouth. Information about local businesses is scattered across maps, social media and personal recommendations, and it is hard for a newcomer, such as a student who has recently moved to Patiala, to judge which listing can be trusted. Existing large marketplaces focus on metropolitan areas and offer limited transparency about where their data comes from.

CityServe addresses this gap with a single web application where users can search local trades by category and locality, contact businesses directly, place orders or service bookings, and leave reviews. Its guiding principle is *honesty by design*: the interface must never imply more trust or more data than actually exists.

1.2 Problem Statement

Local services are (i) **fragmented**, because they are discovered through informal channels; (ii) **hard to verify**, because listings and ratings are rarely labelled by how trustworthy they are; and (iii) **disconnected**, because searching, calling, navigating and booking happen in separate tools.

1.3 Objectives

- Provide a role-based marketplace for customers, vendors and administrators.
- Integrate real Patiala business data into search, category and profile pages.
- Preserve the existing database schema while integrating this data (a non-negotiable project constraint).
- Present data honestly, distinguishing verified partners from directory listings and handling missing ratings and phone numbers explicitly.
- Deliver a polished, responsive, themeable interface with direct contact actions.

1.4 Scope of This Report

The initial prototype has already been presented. This report therefore describes the full system for completeness, but gives priority to the changes made since then (Section 6), the verification of the final build, and the lessons learned. Items that are designed but not yet built are listed explicitly in Section 10.3.

2 Requirements Analysis

2.1 User Roles

The system defines three roles, enforced with role checks in the authentication middleware.

Customer
Browses and searches businesses, places product orders or service bookings, tracks order status, and reviews completed orders.

Vendor

Registers a business, manages products and services, and approves, rejects or completes incoming orders.

Admin

Views all users and vendors, toggles vendor verification, and reads platform analytics.

2.2 Functional Requirements

Table 1: Functional requirements.

ID	Requirement
FR1	Users can register and log in as customer or vendor; sessions use access and refresh tokens.
FR2	Users can browse businesses by category and locality and search by free text.
FR3	Each business has a profile with category, area, contact options, rating and trust label.
FR4	Customers can place an order or booking and follow its status.
FR5	Customers can review a completed order, one review per order.
FR6	Vendors can create and edit products and services and update order status.
FR7	Admins can verify vendors and view analytics.
FR8	The directory must include the 155-business Patiala dataset.
FR9	Users can call a business and open directions with one tap where data exists.

2.3 Non-Functional Requirements

- **Integrity:** no fabricated ratings; missing data must be shown as missing.
- **Compatibility:** zero changes to the existing database schema and zero migration scripts.
- **Performance:** repeated directory lookups must not re-read files or hit the database unnecessarily.
- **Security:** validated input, hashed passwords, token-based authentication, rate-limited authentication endpoints.
- **Resilience:** the application must run without optional third-party keys (maps, payments) using a demo mode.
- **Usability:** responsive layout with light and dark themes.

3 System Architecture

3.1 Technology Stack

3.2 Three-Tier Design

The system follows a three-tier architecture. The frontend (port 3000) renders pages and calls the backend API (port 4000). The backend reads transactional data from MySQL (port 3306) and merges it, at request time, with the Patiala dataset that it holds in memory.

Table 2: Technology stack.

Layer	Technology	Role
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS	Server-rendered pages, client interactivity, theming
Maps	Mapbox GL (with fallback)	Interactive map view; coordinate cards when no token
Backend	Express, TypeScript	REST API under `/api`
Validation & security	Zod, Helmet, rate limiting, JWT	Input checks, headers, abuse control, authentication
ORM & database	Sequelize, mysql2, MySQL 8	Users, vendors, offerings, orders, reviews
Data source	Google Places API (CSV export)	155 Patiala business records
Payments	Razorpay (demo mode by default)	Payment flow without live keys

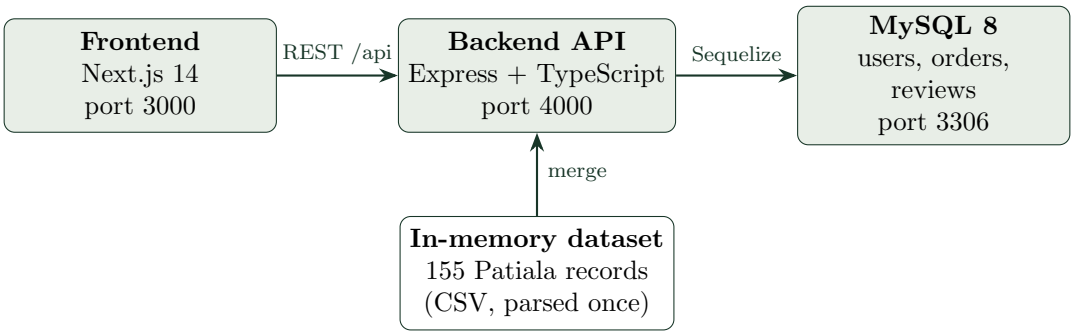


Figure 1: High-level architecture of CityServe.

3.3 Backend Module Structure

The backend source is organised by responsibility: `config` (environment and database), `controllers`, `routes`, `middleware` (authentication, errors, rate limiting), `models`, `services` (dataset engine), `utils`, `validators`, `seed` and `scripts`. Each domain has a matching controller and route module: authentication, vendors, categories, catalog (products and services), orders, reviews, search and admin.

3.4 Data Model

The relational model has six tables. Associations between them are defined centrally in `models/index.ts`.

3.5 Order Lifecycle

An order starts as `PENDING`. The vendor can approve or reject it, an approved order can be completed, and a customer can cancel. Payment status is tracked separately as `PENDING`, `PAID` or `FAILED`. A customer may review an order only after it is completed.

Table 3: Core tables and key attributes.

Table	Key attributes
User	id, name, email, phone, passwordHash, role (CUSTOMER, VENDOR, ADMIN), address, city
Category	id, name, slug, description, icon
Vendor	id, userId, businessName, description, categoryId, type, phone, address, city, latitude, longitude, imageUrl, isVerified, isActive
Offering	id, vendorId, categoryId, type (PRODUCT or SERVICE), name, description, price, unit, stock, durationMinutes, isAvailable
Order	id, orderNumber, customerId, vendorId, offeringId, quantity, type, price, totalAmount, address, scheduledAt, status, paymentStatus, notes, completedAt
Review	id, customerId, vendorId, offeringId, orderId, rating, comment

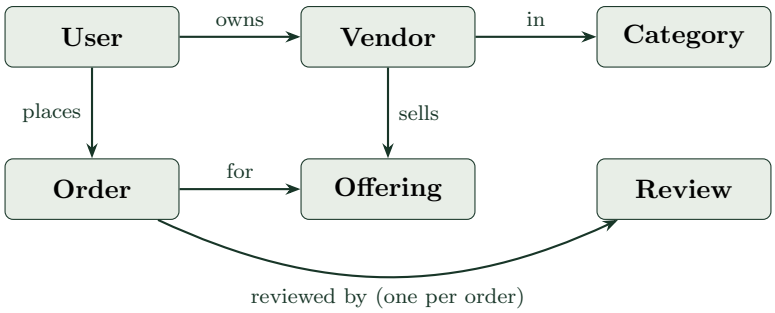


Figure 2: Simplified relationships between the six core tables.

3.6 REST API Overview

3.7 Security Measures

Passwords are stored as hashes. Authentication uses a short-lived access token and a refresh token signed with separate secrets. Role checks (`requireRole`) guard vendor and customer actions. Request bodies are validated with Zod schemas, Helmet sets protective HTTP headers, and rate limiters operate on 15-minute windows, with a stricter limiter applied to the authentication endpoints. Secrets are read from environment variables and never hard-coded in the application logic.

4 Dataset

4.1 Source and Schema

The directory is seeded from `cityserve_patiala_seed_dataset.csv`, a file of 155 Patiala businesses collected through the Google Places API. Each row has the following fields:

```
business_name,category,subcategory,area,city,address,phone,website,latitude,
longitude,rating,review_count,source,source_url,place_id,verification_status
```

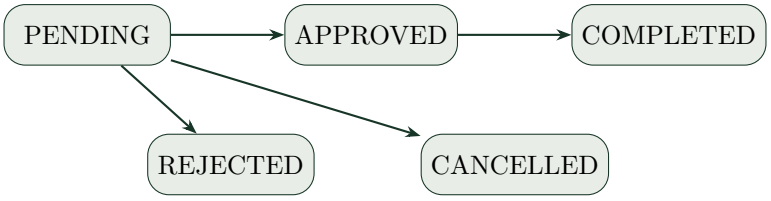


Figure 3: Order status transitions.

Table 4: Main API operations (paths are relative to each module's mount point under `/api`).

Module	Method	Operation
auth	POST	register/customer, register/vendor, login, logout, refresh (auth endpoints are rate limited)
auth	GET, PATCH	me: read and update own profile
vendors	GET	list vendors (merged), vendor by id, own orders
vendors	PATCH	update status of own incoming orders
categories	GET	list categories with counts, category by slug
catalog	GET, POST, PATCH	products and services; create and update restricted to the VENDOR role
orders	GET, POST	create an order, list orders, order by id
reviews	GET, POST	list own and vendor reviews; create (CUSTOMER role only)
search	GET	multi-attribute search
admin	GET, PATCH	users, vendors, analytics, toggle vendor verification

4.2 Coverage and Quality

The statistics show that the dataset is rich enough to serve a real directory but is also incomplete in places. Seven records have no rating and fourteen have no phone number, which directly motivated the data-honesty rules described in Section 6.7. The repository also contains an optional importer script (`npmrunimport:businesses`) that can load the CSV into MySQL. The final prototype does not depend on it: to respect the zero-mutation constraint, the dataset is served from memory.

5 Implementation of the Core System

5.1 Authentication and Authorization

Registration endpoints exist separately for customers and vendors. A vendor registration also creates the vendor profile. Login returns an access token and sets a refresh mechanism; a refresh endpoint renews sessions and a logout endpoint ends them. Middleware extracts the user from the token, and role guards restrict, for example, review creation to customers and offering management to vendors.

Table 5: Listings per category group (20 categories, 155 listings).

Category group	Listings
Laundry / Dry Cleaning	15
Car and Bike Mechanics	10
11 trades (e.g. Electricians, Plumbers, Carpenters, AC Repair, Tutors, Photographers)	8 each
4 trades (e.g. Painters, RO Service)	7 each
Appliance Repair, Pest Control, Cleaning	5, 5 and 4

Table 6: Key dataset statistics.

Metric	Value
Businesses	155
Trade categories	20
Patiala localities	83
Businesses with a rating	148
Mean listing rating (unweighted)	4.82
Total Google reviews behind those ratings	22,119
Businesses without a rating	7
Businesses without a phone number	14

5.2 Catalog, Orders and Reviews

Vendors publish offerings that are either products (with price, unit and stock) or services (with price and duration). Customers place an order or booking against an offering. The vendor then approves, rejects or completes it. Once an order is completed the customer can submit a rating and comment, and the review is linked to the order, the offering and the vendor, which prevents duplicate or anonymous reviews.

5.3 Frontend Pages

The frontend consists of 24 routes. The main customer-facing pages are the homepage, the `/explore` directory with search and filters, category landing pages, vendor profile pages, product and service pages, cart, checkout and order tracking. Authenticated areas include the customer dashboard (orders, profile, reviews), the partner dashboard (orders, products, services) and the admin area (users and partners). Registration, login and support pages complete the set.

5.4 Demo Mode for External Services

Maps and payments depend on third-party keys. When the Mapbox token or Razorpay keys are absent, the application starts in a clearly flagged demo mode: the map view falls back to coordinate cards with Google Maps links, and payments run through a simulated flow. The health endpoint reports both flags so that the state is visible. This allows the full prototype to be demonstrated on any machine.

6 Technical Changes Since the Initial Prototype

This section is the core of the final prototype stage. Table 7 summarises the differences and the following subsections explain each one.

Table 7: Summary of changes.

Area	Initial prototype	Final prototype
Data	Dataset file not used by the backend	155 real Patiala businesses served by an in-memory engine
Backend	Mock vendors, brittle queries	Hybrid controllers merging database and dataset
Look and feel	Generic template, placeholder cards	Editorial design system "The Local Standard"
Ratings	Default 5.0 stars	Honest "Not rated yet" state
Actions	Browse only	One-tap call, directions, map view
Theming	Dark inputs unreadable on some systems	Fixed with explicit `color-scheme`

6.1 Design Constraint: No Schema Changes

The project mandate required zero schema modifications: no columns added, renamed or dropped, no tables created or removed, no `sequelize.sync({alter:true})` or DDL scripts, and no mutation of existing records. All integration work was designed around this restriction.

6.2 In-Memory Dataset Engine

Two mirrored modules implement the engine: `backend/src/services/dataset.service.ts` for the API and `frontend/src/lib/dataset.ts` for server-rendered pages and the directory route handler.

- **RFC 4180 tokenizer.** A hand-written state machine parses the CSV character by character. It handles quoted fields, escaped quotes, commas inside addresses and `\r\n` line endings, which a naive split on commas would break.
- **Singleton cache.** The file is parsed once at server start and the result is reused, so repeated API calls cause no file reads. Cached lookups are reported to complete in under 2 ms.
- **Typed normalisation.** Every row is converted to a `BackendVendorDto` with fields such as id, businessName, categoryId, phone, address, area, coordinates, rating, reviewCount, source, sourceUrl, placeId, verificationStatus, isVerified and isActive. Phone, rating and review count are nullable.
- **Derived indexes.** A locality index with frequency counts (83 localities) and a category index (20 trades, each mapped to an icon) power the filters and category pages.

6.3 Hybrid Backend Controllers

Vendor controller

`listVendors` queries MySQL through Sequelize, filters the dataset by query, category and locality, merges both result sets and places verified partners first. `getVendorById` resolves an identifier by trying the primary key, then the place identifier, then a URL-decoded slug, and finally a normalised business name, so links keep working across both data sources.

Category controller

Merges database and dataset categories and attaches live listing counts (for example, Laundry has 15).

Search controller

Combines `Op.like` matching on offerings and vendors with an in-memory fuzzy match over name, category, subcategory and area.

Each data source is wrapped in a try/catch block so that a missing table or an empty database cannot crash an endpoint; the dataset results are still returned.

6.4 Editorial Design System: The Local Standard

The generic blue template was replaced by a heritage-inspired system. The palette uses Deep Forest, Warm Ivory, Soft Sage, Muted Brass and Charcoal Ink. Headlines use DM Serif Display and interface text uses Inter. Design tokens live in the Tailwind configuration and global stylesheet, and a theme context provides light and dark modes with the choice persisted in the browser.



Figure 4: The Local Standard colour palette.

6.5 Frontend Components and Pages

New or rebuilt components include `HeroSearch` (query, category and a 83-locality dropdown, replacing a cycling placeholder), `VendorCard` (category chip, locality pill, call link and verification badge), `DirectoryMap` (Mapbox GL with coordinate-card fallback), `CategoryCard`, `Navbar` and `Footer`. The `/explore` page offers debounced search (300 ms), category and locality filters with removable chips and a list or map toggle. Vendor profile pages provide a call button and Google Maps directions. A Next.js route handler at `/api/directory` serves lookups and counts without database load.

6.6 Theme Contrast Bug Fix

After the redesign, native form inputs showed dark text on dark backgrounds on systems using a dark operating-system theme. The cause was that browsers colour native controls according to the *operating-system* preference, not the application's own `.dark` class. The fix is two declarations:

```
:root { color-scheme: light; }
```

```
.dark { color-scheme: dark; }
```

6.7 Data Honesty and Integrity Standards

1. **No fabricated ratings.** Earlier versions displayed a default five-star rating on unrated items. The star component now handles null values without producing NaN, and unrated businesses show "Not rated yet".
2. **Trust hierarchy.** *Verified Partner* is reserved for businesses that completed CityServe onboarding. *Directory Listing* is shown on all 155 sourced businesses to make clear that they come from public data.
3. **Graceful gaps.** The 14 businesses without a phone number show a disabled "No phone listed" pill instead of a broken link, and the 7 without a rating omit rating figures.

7 Testing and Verification

7.1 Production Build

The frontend was validated with `nextbuild`. The build compiled successfully, passed linting and type checking, and generated 24 of 24 pages. All routes share 87.3 kB of first-load JavaScript.

Table 8: Selected routes from the production build.

Route	Size	First-load JS
/ (homepage)	2.32 kB	107 kB
/explore	506 kB	611 kB
/vendor/[id]	3.43 kB	108 kB
/category/[slug]	3.63 kB	108 kB
/cart	4.6 kB	101 kB
/admin	2.14 kB	89.4 kB
Shared by all routes	–	87.3 kB

7.2 API Verification

Table 9: Backend API checks.

Request	Observed result
GET/api/health	status ok; payments and maps in demo mode
GET/api/vendors	155 combined listings
GET/api/categories	20 categories with counts
GET/api/search?q=electric	10 or more matching Patiala electrical vendors

These checks were recorded during development. No automated unit or integration test suite exists yet; this is listed as future work.

8 Deployment and Setup

8.1 Prerequisites and Environment

Node.js, npm and a running MySQL 8 server are required. The backend reads its configuration from `backend/.env` and the frontend from `frontend/.env.local`.

Table 10: Environment variables.

Variable	Required	Purpose
`DB_HOST,DB_PORT,DB_NAME,DB_USER,DB_PASSWORD`	Yes	MySQL connection
`JWT_ACCESS_SECRET,JWT_REFRESH_SECRET`	Yes	Token signing (long random strings)
`CLIENT_URL`	Yes	Frontend origin for CORS
`MAPBOX_ACCESS_TOKEN`	No	Maps; demo mode if absent
`RAZORPAY_KEY_ID,RAZORPAY_KEY_SECRET`	No	Payments; demo mode if absent
`NEXT_PUBLIC_API_URL`	Yes	Backend base URL for the frontend

8.2 Running the Application

```
# Backend (port 4000)
cd backend && npm install && npm run dev

# Frontend (port 3000)
cd frontend && npm install && npm run dev
```

Credentials and secrets must be kept out of version control and out of submitted archives.

9 Results and Discussion

The final prototype meets its stated objectives. Real Patiala businesses appear in search, categories and profiles; the database schema is untouched; unrated and phone-less businesses are presented honestly; and users can call or navigate with one tap. Moving the dataset into memory kept the integration non-invasive and fast, and the hybrid controllers allow a business onboarded through the database to appear next to dataset listings in a single, consistently ordered result list.

The trade-off of this design is duplication and scale: the parser exists in both the frontend and backend, and the `/explore` page ships a large client bundle (611 kB first-load JavaScript) because it carries the dataset. At 155 records this is acceptable, but a larger dataset would require server-side pagination or a database-backed index.

10 Conclusion

10.1 Summary

CityServe evolved from an early prototype with generic components and disconnected data into a coherent local services marketplace for Patiala. The final prototype integrates 155 real businesses across 20 trades and 83 localities, introduces an editorial design system, adds direct contact actions, and enforces honest presentation of data, all without altering the database schema.

10.2 Challenges Faced and How We Solved Them

Table 11: Challenges and solutions.

Challenge	Why it was hard	How we solved it
Integrating real data with no schema change	The mandate forbade new tables, columns, migrations and record changes.	Built an in-memory dataset engine and hybrid controllers that merge the CSV with database data at request time.
Messy CSV input	Addresses contain commas and quotes, and line endings vary.	Wrote an RFC 4180 state-machine tokenizer instead of splitting on commas.
Two data sources, one identity	Dataset rows and database vendors use different identifiers.	Normalised every row to a typed DTO and resolved profile links by primary key, place identifier, slug, then normalised name.
Sparse and missing data	7 businesses lack ratings and 14 lack phone numbers.	Made fields nullable end to end, showed "Not rated yet" and "No phone listed", and removed default five-star ratings.
Trust and transparency	Sourced listings could be mistaken for vetted partners.	Introduced distinct Verified Partner and Directory Listing badges.
Unreadable dark inputs	Native controls followed the OS theme, not the app theme.	Set `color-scheme` explicitly for the light and `.dark` themes.
Fragile endpoints	A missing table or empty database could break a response.	Isolated each data source in try/-catch so partial results are always returned.
Missing third-party keys	Maps and payments cannot be demonstrated without credentials.	Added clearly flagged demo modes and a health endpoint that reports them.
Performance of repeated lookups	Re-parsing the CSV per request would be slow.	Parsed once into a singleton cache; added debounced search on the client.
Documentation drift	Older README sections describe a larger earlier data model than the six models in the code.	Treated the source code and the technical changes report as the source of truth and flagged the mismatch for cleanup.

10.3 Limitations

- A recommendation scoring utility (weights: proximity 0.3, rating 0.3, verification 0.2, availability 0.1, price value 0.1) exists in the backend but is not yet wired into ranking.
- The dataset parser is duplicated in frontend and backend, and the `/explore` bundle is large.
- There is no automated test suite for the importer, controllers or user interface.
- Product-type businesses are not supported by the dataset, which contains service businesses only.
- Business owners cannot yet claim an imported listing.
- Project documentation still contains sections from an earlier revision and needs consolidation.

10.4 Future Work

1. Connect the recommendation score to search and category ranking.
2. Add a "Claim this listing" flow so owners can take over directory entries.
3. Introduce automated tests for the parser, controllers and key user journeys.
4. Share one dataset module between frontend and backend and add server-side pagination.
5. Extend the dataset to product-type businesses and additional cities.
6. Enable live Mapbox and Razorpay integrations and consolidate the documentation.

References

[1] Y. Shafranovich, *Common Format and MIME Type for Comma-Separated Values (CSV) Files*, RFC 4180, IETF, 2005.

[2] Next.js Documentation, Vercel. App Router and route handlers.

[3] Express.js Documentation, OpenJS Foundation.

[4] Sequelize Documentation, ORM for Node.js.

[5] MySQL 8.0 Reference Manual, Oracle Corporation.

[6] Zod Documentation, TypeScript-first schema validation.

[7] Google Maps Platform, Places API Documentation.

[8] Mapbox GL JS Documentation, Mapbox.

A Project Structure

```
city-serve-main/
  backend/
    data/cityserve_patiala_seed_dataset.csv
    src/
      config/ database.ts, env.ts
      controllers/ admin, auth, catalog, category, order, review, search, vendor
```

```
    routes/ one route module per controller
    middleware/ auth.ts, error.ts, rateLimit.ts
    models/ User, Category, Vendor, Offering, Order, Review, index.ts
    services/ dataset.service.ts
    scripts/ importBusinesses.ts, syncDb.ts
    seed/ seed.ts
    utils/ jwt, password, distance, recommendation
    validators/ Zod schemas
frontend/
  src/
    app/ pages, dashboards, admin, api/directory route handler
    components/ HeroSearch, VendorCard, DirectoryMap, CategoryCard, Navbar, Footer, ...
    lib/ api.ts, dataset.ts, format.ts, icons.ts
```